

Exercises

Brainster Web Development Academy



Table of contents

- 01 General Info
- 02 Rendering
- 03 Routing
- 04 Tagging and Filtering
- 05 Fixing bugs



Building an application Cookbook

Brainster Web Development Academy



General info

1. HTML & CSS have already been taken care of - any further addition or changes in HTML or CSS should be done via JavaScript
2. You'll only need to write JavaScript
3. Every exercise builds upon the previous one
4. The goal is to have a full-featured cooking recipe application by the end of the exercise.
5. Follow the comments inside `cookbook.js` to keep track of where you should write code.
6. All of the recipe data is already provided, in a variable called `recipeData` (an array of objects).



Re-using functions

1. When working on a project, often we'll run into situation where there are existing functions / classes available for use.
2. In these situations, always try to use the existing code - avoid reinventing the wheel.
3. An example: `createCard()`



Exercise I

1. Using the cookbook.js file:
 - Use the createCard method to draw all recipes on screen.
 - Hint: it's simplest to iterate over all recipes and invoke createCard for each of them.



Routing

1. In its simplest (client-side) form, routing is changing what's on screen based on the browser URL.
2. That solves the problem of seemingly changing the page without an actual page reload.
3. It's faster and feels more interactive.
4. More on what are **Single Page Application** and **Multi Page Application** in next slides



SPA vs MPA Application

1. What is a Single Page Application (SPA)?

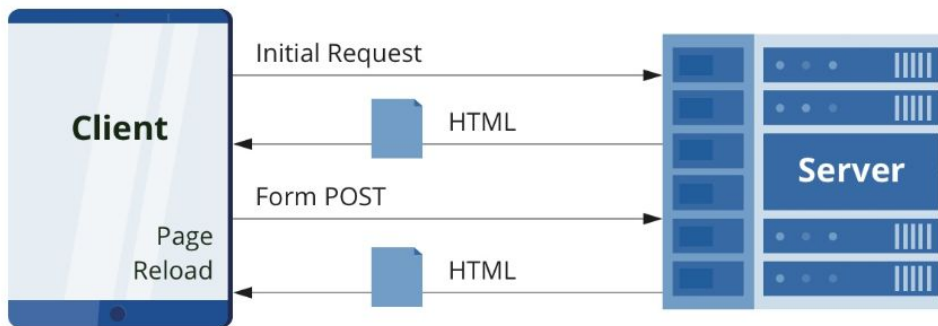
SPA sites consist of only one page, dynamically replacing data on that page instead of serving entirely new pages.

2. What Are Multi Page Application Websites?

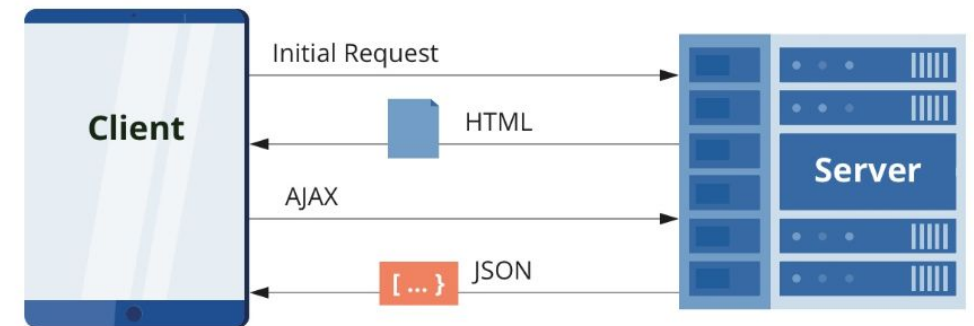
Sites with multiple pages and URLs that have to load every time a user clicks a new link or button

8

Traditional Page Lifecycle



SPA Lifecycle



SPA Benefits

1. Better UX

Site loads initially and the switching between pages does not require reloading of html elements that are already loaded.

2. Back-end / Front-end Separation

Code is separated in server side and client side, meaning that client side does not have to worry about server side and vice versa. Also same backend can be used to serve different client code bases.

3. Caching capabilities

We can effectively cache server data on client side, and save lot's of duplicate requests.



SPA Disadvantages

1. Less SEO opportunities

Site is generated dynamically meaning that ahead of time Search Engine Optimization robots cannot detect content, therefore cannot index the site for SEO rating. In modern frameworks like NEXT.js there is possibility to create SSR (Server Side Rendering) Pages which will overcome this issue. (more on this later)

2. Javascript dependent

SPA applications cannot work without javascript unlike MPA that can generate whole html pages without using JS

3. Initial Loading Time

Initial loading time by default is a bit longer because client needs to download code for whole application even if user is not visiting all pages. Again in modern frameworks like ReactJS, there are multiple solutions for this issue.



MPA Benefits

1. More SEO Opportunities

Due to the fact that multiple pages are generated with content, SEO engines can register and index the content of the pages and assign better ratings.

2. Enhanced Security

Every page goes through the server and validation is more simpler in comparison with SPA

3. Unlimited scalability

MPAs allow you to create new content and place it on new pages. Multi-page apps can include as much information about products or services as required while including lot's of information in SPA will lead to big initial loading time.



MPA Disadvantages

1. Slower Loading Times

While initial loading time of MPA may be faster, than on every other button click or url change there will be another request sent to server to retrieve data.

2. Hard to Maintain

Developers need to maintain each page of a multi-page application separately and regularly



Routing

1. The most basic way to achieve this is by “abusing” the **hashchange** event on the window object:

script

```
window.addEventListener("hashchange", hashHandler)
```

2. The above code will trigger any time the “#” part of the URL changes.



Routing

script

```
function hashHandler(event) {  
  event.preventDefault(); // to prevent the default scrolling behavior  
  let id = location.hash;  
  document.querySelector(`#${id}`).style.display = "block";  
}  
  
window.addEventListener("hashchange", hashHandler);
```



Exercise II

Using the cookbook.js file (in the handleRoute function):

1. Prevent the default behavior;
2. If the hash is empty, render the “home” screen (cardContainer element, use display = ‘flex’);
3. Otherwise, leave the screen empty.



Re-using functions

Second example: `renderRecipe()`



Exercise III

Using the cookbook.js file (in the handleRoute function)

1. Add an “else” clause to the router
2. Hide the `cardContainer` element (use `display='none'`)
3. Show the `fullRecipeContainer` element (use `display='flex'`)
4. Invoke the `renderRecipe` function.



Exercise IV

Using the cookbook.js file

1. Make the router also work on window reloads
Hint: Use the window 'load' event



Exercise V

Using the cookbook.js file

1. We'll also need a way to exit the recipe and go back to the “home” screen (besides the back button in the browser)
2. Add an “a” element in the renderRecipe function (see comments in file).
3. Set its href attribute to “#”.
4. Set its classlist to ('waves-effect', 'waves-light', 'btn', 'orange')
5. Hint: You can add multiple classes via classList add, separated by a comma.
6. Set its text to “Back”
7. Append the new “a” element to the “wrapper” element.



Routing (continued)

- We now have a basic router that switches between two screens - the recipe list and an individual recipe screen with details.
- While not too flexible, this is a good-enough approach to get some quick-n-dirty in-page routing.



Exercise VI

Using the cookbook.js file (renderRecipe, see comments)

1. You probably noticed that we only have instructions, but we lack the ingredients for the recipe.
2. Add a new “h4” element, which will serve as a section title. Set its text to “Ingredients”.
3. Add an ingredient list to the recipe - it should be a “ul” element with a “li” child for each individual recipe ingredient.
4. Append the new “h4” and “ul” element to the “contents” element (see comments for placement).



Exercise VII

Using the cookbook.js file (createCard, see comments)

1. The tags in the recipe list aren't currently clickable - let's fix that :)
2. Add a "click" event listener to every tag element (see comments), which simply logs the tag text to the console (we'll change that later on).



Aside: tagging systems

- Tagging systems are useful for grouping things together and filtering them as needed.
- For the purpose of our exercise, that means listing certain recipes when we click on a tag.
- We can achieve this by a combination of a few things:
 - A way of **filtering** the data in the recipes
 - **A new route to handle the filtering**
 - All while not breaking the existing workflow



Array.filter

- A convenient way of getting a part of the data inside an array is the .filter method.

Array.filter

```
let myArr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

let evenNumbers = myArr.filter(function(element) {
  return element % 2 === 0;
});

console.log(evenNumbers); // [2, 4, 6, 8, 10]
```



Array.filter with objects

- Since our array is made up of objects, when filtering the recipes, we can access each individual recipe property using the “.” (dot) notation. For example

Array.filter

```
let filtered = recipeData.filter(function (recipe) {  
  recipe.tags.includes(theTagWeWant);  
});
```



Exercise VIII

Using the cookbook.js file (continue in the event listener from the previous exercise)

1. Let's first filter the data we need - filter the `recipeData` variable (and create a new variable from it, called `filteredRecipeData`).
2. Each recipe that contains the clicked tag should be a part of the new `filteredRecipeData` variable.
3. Log the result in the console.



Routing (continued)

- We often need to add extra routes to handle new functionality in an application.
- Following the example with `cardContainer` and `fullRecipeContainer`, the process isn't too different - but we'll need a way to tell where we are.
- Hashes can contain `"/` characters too. So, using something like `location.hash = "tags/" + tagName` will do the trick



Aside: splitting strings

- In order to detect in the routing that we're after a "tag" route, we'll need two things.
 - Detecting whether the hash contains the "tag" word (Hint: `location.hash.includes('tag')`)
 - Extracting the actual tag after the "/" part of the hash (`location.hash.split('/')[1]`)



Aside: splitting strings

- For example, if our current hash is “#tag/desserts”

String.split

```
let parts = location.hash.split("/"); // ["#tag", "desserts"]  
let tag = parts[1]; // "desserts"
```

29

- The “tag” variable is exactly what we want to use to filter the recipe array.

String.split - with destructuring

```
const [_ , tag] = location.hash.split("/")  
console.log(tag) // "desserts"
```



Exercise IX

Using the cookbook.js file (else clause of handleRoute)

1. Add an extra nested if which checks whether the current hash includes the word “tag”.
2. If it does, hide `cardContainer` and `fullRecipeContainer`, but set `tagContainer` to `display = “flex”`;
3. Move the main recipe screen to the else clause (it should otherwise remain intact).



Re-using functions (continued)

- If we take a look at `createCard`, we'll realize we can re-use it to show the filtered cards on screen too.
- It takes a recipe as an input argument and outputs the markup we need to show on screen.
- If we were to use that function on the filtered set of recipes, we would achieve exactly that.
- Remember - we can use a function within itself ;)



Exercise X

Using the cookbook.js file (back to the click event handler for tags)

1. Set the current hash to 'tag/' + the current tag
(hint: just like we can get location.hash, we can also set it by using location.hash = ...)
2. Remove the existing contents of the `tagContainer` (hint: Use `.innerHTML = ""`).
3. Iterate over `filteredRecipeData`, and invoke the `createCard` function for each of the recipes. Append the new card returned from `createCard` to the `tagContainer`.



Exercise XI

Using the cookbook.js file

1. We still need a way to return to the home screen - and we have something that tries to look like a logo on the top.
2. Add a click event listener on the logo (header element), which sets location.hash to "" (returning the application to the home screen).



Fixing bugs

- You probably noticed that we have a bug of sorts - when we hit a “tag/” route and reload, the screen remains empty.

Let's fix that.

- Currently, the tag routes only render on a click event. It's a bit more maintainable to keep all of that functionality in a single place. Let's move the entire logic for that to the `handleRoute` function, since it should be the only place where screen changes are handled.



Fixing bugs

- Often, this means moving code around. And we'll do exactly that - simply move the existing parts to their appropriate place, in order to have a consistent experience when we reload the window.



Exercise XII

Using the cookbook.js file

1. Parse the current tag from location.hash in the `handleRoute` function (where we show the `tagContainer`, in the nested if clause).



Exercise XIII

Using the cookbook.js file

1. Move the recipe filtering logic from the tag “click” event handler to the router (just below the code from last exercise).



Exercise XIV

Using the cookbook.js file

1. Move the recipe rendering logic (the `createCard` part) from the tag “click” event handler to the `handleRoute` function (just below the previous exercise). Don’t forget the `innerHTML` reset!

Hint: The only code remaining in the click event handler should be the `location.hash = 'tag/'+tag` part.



Continued...

We reached so far, we will continue building application next class

